

Evaluation of a set of functions related to Ripley's K analysis

Gabriel Arellano

2026-06-01

Contents

1	Goal	1
2	Load code	2
3	Evaluation	3
3.1	Specific functions	3
3.2	Is there a need for this?	4
3.3	Generalization?	4
3.4	Conclusion	4
4	Design principles for alternative code	5
5	Current status of alternative code	6

1 Goal

Evaluate, understand, and decide over a set of related functions that count individuals in circular areas and compare those counts with a null expectation, implementing Ripley's K test. At the moment, this is an isolated set of functions; see "1-dependencies-and-priorities-2026-05-20.pdf" report.

This set contains the following functions:

- `CalcRingArea` in `spatial > RipUvK.r`
- `partialcirclearea` in `spatial > RipUvK.r`
- `circlearea` in `spatial > RipUvK.r`

- `Annuli` in `spatial > RipUvK.r`
- `NDcount` in `spatial > NeighborDensityFun.r`
- `NeighborDensities` in `spatial > NeighborDensityFun.r`
- `RipUvK` in `spatial > RipUvK.r`

These functions are located in two different scripts but all are related to the same type of calculation: counting number of neighbors in a given area, and compare that with some null expectation (Ripley's K function). The set covers all code in `RipUvK.r` and `NeighborDensityFun.r` original scripts.

2 Load code

```
# This is my local path. You may have to adapt this.
g <- regexpr("fgeo2", getwd())
path <- substr(getwd(), 1, g + attr(g, "match.length"))

# This chunk of code will source all the old R scripts:
f1 <- list.files(paste0(path, "01_utilities/R/old/"),
                 full.names = TRUE, pattern = "\\R$", ignore.case = TRUE)
f2 <- list.files(paste0(path, "02_biomass/R/old/"),
                 full.names = TRUE, pattern = "\\R$", ignore.case = TRUE)
f3 <- list.files(paste0(path, "03_spatial/R/old/"),
                 full.names = TRUE, pattern = "\\R$", ignore.case = TRUE)
f4 <- list.files(paste0(path, "04_abundance_diversity/R/old/"),
                 full.names = TRUE, pattern = "\\R$", ignore.case = TRUE)
f5 <- list.files(paste0(path, "05_demography_growth/R/old/"),
                 full.names = TRUE, pattern = "\\R$", ignore.case = TRUE)
f6 <- list.files(paste0(path, "06_topography_habitat/R/old/"),
                 full.names = TRUE, pattern = "\\R$", ignore.case = TRUE)
f7 <- list.files(paste0(path, "07_mapping_plotting/R/old/"),
                 full.names = TRUE, pattern = "\\R$", ignore.case = TRUE)

ff <- c(f1, f2, f3, f4, f5, f6, f7)
for(f in ff) source(f)

# Use this to see the source code, manually
if(FALSE)
{
  CalcRingArea
  partialcircleara
```

```

circlearea
Annuli
NDcount
NeighborDensities
RipUvK
}

```

3 Evaluation

3.1 Specific functions

Some functions work on areas, other functions work on counts.

The function `circlearea` is trivial.

The `partialcirclearea` makes edge corrections considering the dimensions of the plot. That will be important in forest applications. It seems the most interesting piece of functionality. Apparently Condit implemented Goreaud & Pélissier (1999, Journal of Vegetation Science 10: 433-438), which are analytical solutions for circular and rectangular study areas. I consider keeping it, although it is very cryptic and virtually impossible to “touch”.

`CalcRingArea` seems to be just the accumulated sum of (possibly truncated) circles, one for each point in a dataset. Note:

- it calculates areas of circles, not rings, despite its name
- it does not correct for overlap, so it counts many times the same areas

The area of rings (“annuli”) is calculated by the `Annuli` function, which repeatedly calls `CalcRingArea` and subtracts small circles from larger circles successively. This general approach should be kept.

`NDcount` and `NeighborDensities` try to do similar things. `NDcount` (author Tania Brenes) wraps `splancs::khat` function to count neighbors with edge correction. `NeighborDensities` (author Rick Condit) incorporates filters, the possibility of counting neighbors from a different dataset (e.g. adult trees neighboring seedlings), and “counting” as accumulated basal area instead of individual counts. It depends on specific column names like “status”. There are risky unit conversions inside, it does make strong assumptions about the units of the input: clearly a vulnerable point.

`RipUvK` is the highest-level function in this group of functions. It runs over each species calculating a series of spatial statistics for con-specifics at different distances. It seems easy to use, it is not very long. It is not very flexible, and calls external functions of undefined packages, probably `splancs` package: `khat`, `spoints`.

3.2 Is there a need for this?

Yes. Calculations over neighborhoods are widely used in ForestGEO / ATFS, for example to estimate competition around target individuals and so on. Perhaps not to much for point pattern analyses, as it has been known for a long time that almost all forest species show aggregated patterns across scales. But, in general, counting points in certain areas around target individuals (or locations) is valuable. There are many questions that can be approached that way, if the code has flexibility.

Modelling or comparison with null expectations is less valuable for the `fgeo2` package. The Ripley's K function is surely maintained by other packages: `spatstat::Kest` and `spatstat::Lest` may be the gold standard (and fastest?). The code implements a well-known mathematical formula so it should not require a lot of maintenance. However, the `RipUvK` function is a clear candidate to remove, and substitute by a more general tutorial on how to apply Ripley's K to forest data.

3.3 Generalization?

The study of neighborhoods can be done in many different ways, so it would be important to maximize flexibility in things like selecting the focal trees, picking neighborhood-level metrics, etc. If such flexibility is not feasible to incorporate within `fgeo2` code, then it should be left out the functions, and substitute by examples or guidelines that the user can apply to their case and goals.

Perhaps most of these functions can be incorporated into a more general function that counts (or summarizes) data belonging to different classes, within areas that have a minimum and a maximum radius.

Perhaps generalize rings and circles by declaring minimum and maximum radius, with minimum radius = 0 in all cases? Although that would allow overlapping rings and that may cause problems. I am not sure why circles vs. rings are important but the distinction is sharp, probably no need to make those instances of more general geometry.

3.4 Conclusion

Because the code has obvious points of vulnerability (units, column names, ...), and has very limited flexibility (filters, aggregation, expectations), I decide to rewrite.

4 Design principles for alternative code

Ideally, replacement code should incorporate the flexibility to count neighbors of any class to any class, for example: adult tree neighboring seedlings, individuals of species A neighboring species A (conspecifics), individuals of species A neighboring species B (some heterospecifics), etc. The conspecific vs. heterospecific counting, which is the most frequent, is just a special case. But parameters should be general, not specific assuming this particular application.

It should be general and count everything in the input. If the user wants to focus on trees that are alive, for example, they should subset the dataset first. It does not seem robust to filter from within the function and make assumptions about column names, etc. Filter outside gives all the flexibility to the user, even if they keep lianas and trees together, or whatever they want. This is aligned with the goal of reducing dependency from “orthodox CTFS format”.

Absence of strong defaults about status may help people think a bit about things that happen around dead trees, or how trees recruit around certain locations, and so on. It is better to have inputs that are simply pre-filtered (x, y) coordinates instead of “data” tables to be processed within the function. This can facilitate studying things around locations in a general sense, not just trees.

Edge effects are important. Original code handles rectangular boundaries only. Ideally, we should be able to define edges of any arbitrary shape. For example, so Ngel Nyaki plot can truncate circles to the border of the forested part of the plot, not just the rectangular borders. Or do counts + areas for specific habitat types within the rectangular plot, etc. Goreaud & Péliissier (1999) proposed an algorithm to generalize their approach to arbitrary boundaries, but implementing that would involve “fastidious calculations” (as they say) and would be prone to error. It feel potentially powerful but difficult to write and virtually impossible to debug. Errors in implementation will be almost impossible to detect. A grid-based approach is closer to brute force but is easier to code and check. It should be preferred if computational costs allow it. We need ways of adjusting grid resolution as a function of tolerable error.

Calculating distances to focal points could be relatively easy, but memory and computation is important for large datasets. Almost surely this problem has been solved for large datasets in other packages.

It should be unit agnostic: return everything on the same units as the input. This is a general criterion for code update.

Ideally, the code should be fast. Solutions without loops would be preferred. Memory could be a constrain so tutorials should teach how to do things by chunks.

Ideally, we want to separate the identification of neighbors (who is neighbor of whom) from the counting. Counting is just a metric of abundance, but we could use accumulated basal area or biomass, etc. Both steps are somehow mixed in the current code, limiting flexibility.

Comparing with expectations: the current code uses Ripley's K function, which is an statistic that measures the deviation of a given count in a given area relative to the null expectation of random placement (Poisson point process or PPP). This is a well-known technique implemented in many other packages. It should be deprecated and substituted by tutorials on how to apply it. The tutorials can also show how to implement numerical null models to compare observed counts (or abundances) of neighbors around focal trees with the expectations. Numerical null models would have clear advantages:

- They would not be limited to individual counts as now, but could incorporate observations vs. expectations for other metrics, such as accumulated basal area, total biomass, etc.
- The user could model, potentially, other null expectations, not just PPP.
- If the null model is constrained to a given boundary, it would reflect edges, so deviations from the model would solve for edge effects directly.

In any case, numerical null models require fast code, because of computational costs. Because these are very flexible tools, they should not be hard-coded in `fgeo` functionality.

If all these things are implemented, an updated workflow would expand the current feasible workflow:

- allow the use of any metric of abundance, maybe any characterization of neighborhoods
- neighbors of any class around focal points of any class, defined outside
- filters are outside: maximum flexibility (and responsibility)
- minimize assumptions: work on $\{x, y\}$ general datasets, with boundaries expressed as spatial objects
- account for edge effects in boundaries of arbitrary shape
- allow the use of any null distribution of points, not just PPP

5 Current status of alternative code

All the improvements above have been implemented. Things that require a lot of flexibility are outside the functions, substituted by tutorials or guidelines. The new functions focus on the common operations only:

- `get_distances_up_to_r()`: returns $\{i, j, \text{distance between } i \text{ and } j\}$, which is the most general and flexible way of identifying neighbors within a given distance r . The user can then process these neighborhoods any way they want.
- `get_boundary()`: returns an `sf` polygon object with a boundary of any arbitrary shape. It will be passed to the other functions so they can process arbitrary boundaries.
- `get_radii()`: implements useful defaults and rules of thumb to define the sequence of scales to evaluate, both for circles and rings.

- `core_vs_edge()`: an auxiliary function to split a set of points into far or near a given boundary. It is used to accelerate grid-based calculation of areas.
- `get_area_of_neighborhoods()`: calculates the available area of each neighborhood, which is the most direct way of controlling for edge effects. It uses exact formulas for areas of complete circles (or rings) when the neighborhood is far from the boundary, but a grid-based approach for arbitrarily-shaped neighborhoods that intersect with an arbitrarily-shaped boundary. The density of the grid used is adjusted to meet a user-defined tolerance to error (5% by default).

Filtering, neighborhood summarizing, and comparison with expectations, are outside the functions to allow the user maximum flexibility.

See “neighborhoods.qmd” in the spatial module.